

TECHNICAL REPORT

Kota: Engineering an Antifragile, Self-Compiling Autonomous Agent

Erick Oduniyi
Intelligent Interfaces Group
eeoduniyi@gmail.com

July 2026

Abstract

Modern Large Language Model (LLM) scaffolding treats the agent as a stateless input-output function, representing a loop that re-evaluates context on every query. We present **Kota**, a local-first autonomous agent whose design is guided by the thesis that software loops, electrical circuits, and neural circuits share the same structural basis: mechanisms of transformation and exchange operating at different physical scales. Building on this claim, Kota is engineered with five interlocking systems: (1) an asynchronous Rust event bus decoupling inference from execution; (2) a modular *skill tree* forming the basis of a dynamic Option Keyboard for compositional prompt synthesis; (3) a two-tiered hierarchical memory system via Turso/libSQL implementing *Mental Time Travel*; (4) a Python-driven evolutionary Meta-Harness with a Human-in-the-Loop (HITL) gateway and Energy Score for resource-aware selection; and (5) an *LLM Wiki* that accumulates compiled knowledge incrementally rather than retrieving raw documents at query time. Kota is designed to be polyglot, open-ended, and adaptive; it is not optimized for a single benchmark, but designed to grow stronger through use.

Contents

1	Introduction	2
1.1	Design Philosophy	3
1.2	Contribution Overview	3
2	The Rust Agent Core	3
2.1	The Event Bus	3
2.2	The Agent Loop	4
2.3	Tool Execution	5
2.4	Recursive Agent Delegation	5
2.5	The TUI and Remote Web Controller	5
2.6	Software Construction Principles	5
3	The Skill Tree and Option Keyboard	6
3.1	The Skill Tree	6
3.2	The Option Keyboard	6
3.3	Go-Based High-Performance Auxiliary Processes	7

4	Hierarchical Memory and Mental Time Travel	8
4.1	Turso/libSQL as the Memory Substrate	8
4.2	Episodic Recall via Sparse Keyword Retrieval	8
4.3	Working Memory vs. Episodic Memory	9
5	The Evolutionary Meta-Harness	9
5.1	The Evolution Loop	9
5.2	The HITL Gateway	10
5.3	The Energy Score	10
5.4	Multi-Model Support	10
5.5	Continual Learning Ensembles	10
5.6	Agent-Native Documentation (OpenWiki)	11
6	The LLM Wiki: Persistent Knowledge Compilation	11
6.1	Architecture	11
6.2	Operations	12
6.3	The Index and Log	12
6.4	Relation to Vannevar Bush’s Memex	12
6.5	External Vault Support	12
7	Cognitive RF Security: Sensing, Diversity, and Information Theoretic Resilience	12
7.1	MIMO Prompting and Spatial Diversity	13
7.2	Cognitive Context Sensing	13
7.3	Markov Channels and Context Capacity	13
8	On Loops, Circuits, and Antifragility	14
9	Discussion and Future Work	14
10	Conclusion	15

1 Introduction

The standard LLM agent paradigm is a loop: a user submits a message, the LLM produces a response (possibly with tool calls), the tool results are appended, and the cycle repeats. This is a **for** loop. It shares its topology with a simple electrical circuit: a voltage source drives current through a resistive load, doing work, and returning. Both are *circuits*: energy enters, transformation occurs, and the state of some downstream element is changed.

This analogy is supported by structural parallels. In electrical engineering, an ideal transformer changes the impedance of a load without altering its power budget, stepping voltage up or down through magnetic coupling. In machine learning, the Transformer architecture [1] performs an analogous operation: it changes the *representational* scale of an input sequence through attention, without altering the fundamental information content. In science fiction, a Transformer *converts* between forms; for example, a car becomes a robot. The common characteristic is conversion: energy, information, or form, exchanged across a boundary.

Kota is designed around this recognition. Every component, from the Rust async event bus to the SQL episodic memory to the Python evolutionary harness, is a circuit element performing a specific conversion. The system as a whole is an orchestrated array of such elements. **Its**

robustness arises from the architecture, not from any individual element’s robustness.

1.1 Design Philosophy

Kota’s guiding principles, encoded in its `base system prompt` from the initial commit:

- **Polyglot & Plug-and-Play.** The agent is language-agnostic. It selects the best language for the task, such as Rust for performance, Python for data science, or Go for networking, without preference.
- **Speed, Efficiency, & Simplicity.** Minimal token generation; prune unnecessary dependencies; prefer `green execution paths`.
- **Security & Moderation.** `No code that sabotages, poisons, or spies`. Local privacy is absolute.
- **Energy & Sustainability.** Treat computation as a physical resource. `Optimize for compute efficiency, not just correctness`.

These principles are not aspirational; they are operationalized through the `Energy Score` in the `Meta-Harness` (Section 5) and the `HITL` review gateway.

1.2 Contribution Overview

This paper makes the following engineering and research contributions:

1. A detailed account of the **Rust agent core** and its asynchronous event-driven architecture (§2).
2. A description of the **skill tree** and the Option Keyboard mechanism for dynamic prompt synthesis (§3).
3. The **Turso/libSQL hierarchical memory system** implementing mental time travel (§4).
4. The **evolutionary Meta-Harness** with HITL, Energy Score, and Continual Learning Ensembles (§5).
5. The **LLM Wiki** pattern for persistent, compounding knowledge compilation (§6).

2 The Rust Agent Core

Kota’s runtime is written in Rust. The choice is intentional: Rust’s ownership model eliminates entire classes of memory safety bugs, its async runtime (`tokio`) enables massively concurrent I/O without a garbage collector, and its type system enforces correct state machine transitions at compile time.

2.1 The Event Bus

The central design pattern is a *typed event bus* implemented with `tokio::sync::broadcast`. Every action the agent takes emits a structured `AgentEvent`:

```
1 pub enum AgentEvent {  
2     UserMessage { text: String },  
3     StepStarted { step: usize, tokens_in: usize },  
4     Token { text: String },  
5     Thinking { text: String },  
6     ToolCallStarted { step: usize, tool: String, args: serde_json::Value },
```

```

7   ToolCallFinished { step: usize, tool: String, duration_ms: u64,
8                       success: bool, result_preview: String },
9   Done { step: usize, total_tokens: usize, duration_ms: u64 },
10  BudgetWarning { used: usize, max: usize },
11  Error { message: String },
12 }

```

Listing 1: The AgentEvent enum (src/events.rs). Every state change is an observable, typed event.

This design is the direct application of an Event-Driven Architecture pattern: components are decoupled via the message bus, and multiple subscribers can observe the same event stream independently. The TUI renderer, the benchmark logger, the remote WebSocket server, and the HITL gateway all subscribe to the same bus without any coupling between them.

2.2 The Agent Loop

The core `Agent::process` method implements a standard ReAct-style [2] agentic loop, but with several critical engineering decisions:

```

1  pub async fn process(&mut self, user_input: &str,
2                      tx: broadcast::Sender<AgentEvent>) -> anyhow::Result<()>
3  {
4      // 1. Episodic Memory Recall (Mental Time Travel)
5      let keyword = user_input.split_whitespace()
6                          .max_by_key(|w| w.len()).unwrap_or("");
7      if keyword.len() > 4 {
8          if let Ok(memories) = self.memory.query_episodic_memory(keyword, 2).
9      await {
10         if !memories.is_empty() {
11             // Silently inject distant memory into current context
12             self.messages.push(Message { role: "system".into(),
13                 content: Some(format!("EPISODIC MEMORY RECALL: {}", ...))
14         }, ..
15         });
16     }
17 }
18 // 2. Context budget guard (approx. 4 chars/token)
19 let approx_tokens: usize = self.messages.iter()
20     .map(|m| m.content.as_ref().map_or(0, |c| c.len() / 4)).sum();
21 if approx_tokens > self.max_tokens * 3 / 4 {
22     let _ = tx.send(AgentEvent::BudgetWarning { used: approx_tokens,
23         max: self.max_tokens });
24 }
25 // 3. Stream LLM response, execute tool calls, loop
26 loop {
27     self.llm.chat_stream(messages, tools, |event| { ... }).await?;
28     if !pending_tool_calls.is_empty() {
29         // Execute tools, append results, continue loop
30     } else { break; }
31 }
32 Ok(())
33 }

```

Listing 2: Simplified agent loop in src/agent.rs. The loop continues until the model produces no tool calls.

The budget guard is a direct implementation of the Energy philosophy: the agent warns before it exhausts its compute budget, giving the TUI time to signal the user.

2.3 Tool Execution

Kota exposes six primitive tools to the model, composable to perform any filesystem or shell operation:

Tool	Args	Effect
<code>read_file</code>	<code>path</code>	Read a file from the working directory
<code>write_file</code>	<code>path, content</code>	Write/overwrite a file
<code>list_dir</code>	<code>path</code>	List directory entries
<code>run_command</code>	<code>command</code>	Execute a shell command
<code>search</code>	<code>pattern, path</code>	Grep for a pattern in files
<code>fetch_news</code>	<code>query</code>	Query a live news API
<code>delegate_task</code>	<code>task, mode</code>	Delegate a sub-task asynchronously to a nested agent

Table 1: Kota’s primitive tool set. Composed sequentially, these enable complex multi-step tasks.

2.4 Recursive Agent Delegation

To mitigate context window inflation and prevent “context rot” when executing complex long-horizon tasks, Kota implements a *Recursive Agent (RA)* orchestration pattern. Rather than executing all sub-steps sequentially inside a single, increasingly bloated working memory context, the parent agent can invoke the `delegate_task` tool.

When invoked, the runtime dynamically instantiates a child **Agent** with a cloned LLM client handle, the same working directory, and the target **AgentMode** requested by the parent (e.g., **Librarian** or **Coder**). The child agent is run to completion asynchronously in a **nested event loop**. To provide real-time visibility without inflating the parent’s token count, the parent **spawns a background task that listens to the child’s token stream and forwards it to the parent’s event bus formatted with a `Sub[mode]>` prefix**. Once completed, only the final synthesized response of the child is returned to the parent as the tool output. This **modular, tree-structured delegation** isolates the child’s intermediate trajectories, keeping the parent’s context clean and preserving performance on deep reasoning tasks.

2.5 The TUI and Remote Web Controller

Kota ships with two simultaneous interfaces: a terminal UI built with `ratatui` and a WebSocket-backed remote HTML controller served by `axum`. The TUI renders the ASCII banner, live streaming tokens, active tool call indicators, step counts, and agent mode. The remote controller, served at `localhost:8765`, provides the same interface in a browser, which is useful for pairing sessions or running the agent headless on a server.

The TUI state machine is straightforward: a Rust **App** struct holds a scroll offset, input buffer, output history (tagged with `LineKind` variants), and the current **AgentMode**. A `crossterm` keyboard listener runs in a separate task and sends keystrokes via `mpsc` to the rendering loop.

2.6 Software Construction Principles

An important engineering decision embedded in Kota’s earliest commits is the application of MIT 6.102 Software Construction principles [9] directly into the agent’s core instructions. Kota is not only designed by these principles, but it also *teaches* them to any language model it wraps:

- **Specifications & ADTs.** Define preconditions and postconditions. Design safe Abstract Data Types avoiding representation exposure.
- **Rep Invariants.** Maintain strong representation invariants, especially for concurrent data structures.
- **Concurrency.** Write safe concurrent code using message-passing or strict mutual exclusion, avoiding shared mutable state.
- **Static Checking & Immutability.** Maximize compile-time guarantees and functional patterns.

This creates a **virtuous loop**: Kota’s Rust runtime enforces these principles at the compiler level, and the agent’s system prompt enforces them at the code generation level.

3 The Skill Tree and Option Keyboard

The most significant architectural evolution in Kota’s history (commit 6646d42) was the **decomposition of a monolithic system** prompt into a modular *skill tree*, which is a directory of independent Markdown files, each encoding the values, behaviors, and knowledge of a specific professional role.

3.1 The Skill Tree

The `.kota.skills/` directory contains six foundational skills:

- **base.md:** The core philosophy shared by all modes: polyglot, energy-aware, privacy-first, Git-disciplined.
- **coder.md:** Staff Software Engineer persona. MIT 6.102 construction principles, full-stack testing, ADTs.
- **research.md:** Interdisciplinary Research Scientist. Physics, linguistics, brain sciences, rigorous cross-domain synthesis.
- **eval.md:** Principal Security Evaluator. Red-teaming, taint analysis, quantitative vulnerability metrics.
- **cpe.md:** macOS/Windows Client Platform Engineer. Fleet management, launchd, MDM, osquery telemetry.
- **architect.md:** Principal Distributed Systems Architect. Load balancing, sharding, CDN, event-driven patterns.
- **librarian.md:** LLM Wiki Compiler. Append-only log, index maintenance, contradiction resolution.

3.2 The Option Keyboard

Inspired by the RL work of Barreto et al. [3], which showed that a linear combination of primitive policy *options* can synthesize new compound behaviors without retraining, Kota applies this principle to prompt engineering. The `SkillComposer` (commit f3e3d5b) treats each skill file as a basis vector in a space of agent behaviors:

```
1 pub fn compose(&self, weights: &HashMap<&str, f32>) -> String {
2     let mut sorted_weights: Vec<(&str, &f32)> = weights.iter()
3         .map(|(k, v)| (*k, v)).collect();
4     sorted_weights.sort_by(|a, b| b.1.partial_cmp(a.1).unwrap());
```

```

5
6     let mut composed_prompt = String::new();
7     for (skill_name, weight) in sorted_weights {
8         if *weight > 0.0 {
9             let skill_content = self.load_skill(skill_name);
10            if !skill_content.is_empty() {
11                composed_prompt.push_str(&format!(
12                    "## {skill_name} (weight: {weight:.1})\n{skill_content}\n
13\n"
14                ));
15            }
16        }
17    }
18    composed_prompt
19 }

```

Listing 3: SkillComposer applies weighted composition to skill basis vectors (src/skills.rs).

The mode-to-weight mapping encodes the Option Keyboard chords:

Mode	Skills (weight)	Behavioral Intent
Coder	coder (1.0), eval (0.2)	Engineer who also red-teams their own code
Research	research (1.0), coder (0.3)	Scientist who writes empirical validation scripts
Cpe	cpe (1.0), architect (0.4)	Platform engineer who reasons about system design
Architect	architect (1.0), cpe (0.5)	Systems designer grounded in platform realities
Eval	eval (1.0)	Dedicated safety evaluator
Librarian	librarian (1.0), research (0.4)	Knowledge compiler drawing on research synthesis skills

Table 2: Option Keyboard chords: each mode is a weighted linear combination of primitive skill basis vectors.

Crucially, modes are hot-swappable at runtime. The TUI intercepts the `/mode <name>` command, reconstructs the system prompt by invoking `SkillComposer.compose`, and injects the new prompt without resetting the conversation history. This allows the agent to transition mid-task; for example, from **Research** to **Coder** to implement a hypothesis that emerged during literature review.

3.3 Go-Based High-Performance Auxiliary Processes

To leverage the polyglot design philosophy, Kota employs a suite of compiled Go-based auxiliary tools to handle CPU-bound or external systems-monitoring tasks. By offloading these helper processes to a language optimized for static compilation and concurrent runtime execution, the core Rust agent remains unburdened. Three Go services are integrated:

1. **Wiki Search Engine** (`search.wiki.go`): Performs multi-threaded directory walks and keyword searches over the compiled markdown files in `.kota.knowledge/`. It uses Go’s native goroutines and sync primitives to parallelize file access, returning structured JSON search results to the Rust agent loop.
2. **Resource Watchdog Daemon** (`watchdog.go`): A lightweight service listening on port 8766 that exposes a REST API returning system status, memory allocation, and active VRAM/GPU proxy metrics. This allows Kota to evaluate hardware constraints before spawning heavy evolutionary or reasoning workloads.
3. **Agent Coordinator** (`coordinator.go`): A management CLI to spawn Kota agent

subprocesses and monitor watchdog status, acting as a gateway for multi-agent coordination.

4 Hierarchical Memory and Mental Time Travel

The most fundamental failure mode of LLM agents is *context degradation*: the model has no persistent memory across sessions, and even within a session, the effective range of attention degrades as the context grows. Long contexts are also computationally expensive; *processing n tokens costs $O(n^2)$ in naive attention*, the quadratic scaling that drives the energetic cost of “alchemy” to impractical levels.

Kota’s hierarchical memory system (commit 9beb238) addresses this through the Mental Time Travel framework of Lampinen et al. [4]: rather than expanding the context window, the agent recalls *compressed episodic traces* from a persistent database.

4.1 Turso/libSQL as the Memory Substrate

Kota uses **Turso**, a distributed SQLite service built on libSQL, as its memory backend. The choice reflects the philosophy of starting local (SQLite) while preserving the path to global scale (Turso’s edge distribution). libSQL’s MVCC (Multi-Version Concurrency Control) prevents write contention in concurrent agent sessions. The **MemoryStore** schema:

```
1 CREATE TABLE IF NOT EXISTS conversations (
2     id TEXT PRIMARY KEY,
3     mode TEXT NOT NULL,
4     created_at TEXT NOT NULL
5 );
6 CREATE TABLE IF NOT EXISTS messages (
7     id INTEGER PRIMARY KEY AUTOINCREMENT,
8     session_id TEXT NOT NULL,
9     role TEXT NOT NULL,
10    content TEXT NOT NULL,
11    created_at TEXT NOT NULL
12 );
```

Listing 4: Memory schema in src/memory.rs. Conversations and messages are linked by session_id.

Every message in every session is persisted immediately after it is produced or received. This ensures that no reasoning is lost even on a hard crash.

4.2 Episodic Recall via Sparse Keyword Retrieval

When the user submits a message, the Rust agent performs a lightweight linguistic analysis: it extracts the longest token from the input (a heuristic for the most semantically dense word), then queries the **messages** table using a SQL LIKE predicate across all historical sessions:

```
1 pub async fn query_episodic_memory(&self, keyword: &str, limit: usize)
2     -> anyhow::Result<Vec<String>> {
3     let pattern = format!("%{}%", keyword);
4     let rows = self.conn.query(
5         "SELECT content FROM messages WHERE content LIKE ?1
6         ORDER BY created_at DESC LIMIT ?2",
7         libsql::params![pattern, limit as i64],
8     ).await?;
9     // Collect and return matched memory fragments
10 }
```


Matched memory fragments are injected into the conversation as a silent `system` message prefixed with `EPISODIC MEMORY RECALL`, making distant context available without forcing the model to hold all prior sessions in its working memory. **The mechanism is structurally equivalent to the hippocampal-neocortical memory consolidation process in biological cognition: sparse, compressed traces from past experience are reactivated when a contextually similar cue is detected.**

4.3 Working Memory vs. Episodic Memory

	Working Memory	Episodic Memory
<i>Substrate</i>	In-process Vec<Message>	Turso/libSQL SQLite
<i>Scope</i>	Current session only	All past sessions
<i>Retrieval</i>	Full linear scan (in-context)	Sparse keyword LIKE query
<i>Capacity</i>	Bounded by <code>max_tokens</code>	Unbounded (disk-backed)
<i>Latency</i>	Zero (in-memory)	Single SQL round-trip
<i>Persistence</i>	Ephemeral	Durable (MVCC)

Table 3: The two tiers of Kota’s hierarchical memory system.

5 The Evolutionary Meta-Harness

Kota is not a static system. Its capabilities are continuously improved through a Python-based evolutionary harness (`meta_search/meta_harness.py`) that treats the skill tree as an evolvable genome.

The harness descends from the KIRA agent evolutionary framework developed on Terminal-Bench 2.0 (TB2), which is a benchmark of diverse long-horizon computer-use tasks. Kota adapts this loop to evolve the *content* of the `.kota_skills/` Markdown files rather than the agent’s code structure, keeping the core runtime stable while the behavioral surface area grows.

5.1 The Evolution Loop

The harness runs a series of phases per iteration:

1. **Propose.** A Claude session reads the current skill tree, the evolution history (`logs/evolution_summary.json`) and proposes a mutation: growing a skill (adding new capabilities), pruning a skill (removing costly behaviors), or editing an existing skill.
2. **Evaluate.** The mutated agent is evaluated on N TB2 tasks across K trials using a Harbor job runner. Pass rates per task are computed.
3. **HITL Review.** Before the mutation is admitted to the frontier, a human arbiter reviews it (Section 5.2).
4. **Update Frontier.** Approved mutants update the frontier: a JSON file mapping task categories to the historically best-performing skill tree configuration.
5. **Phase Final.** The current best generalist agent is evaluated across 5 trials. The Continual Learning Ensemble is simulated (Section 5.5).

5.2 The HITL Gateway

A critical design decision introduced in commit 6646d42 is that no skill tree mutation is ever admitted automatically. The HITL gateway intercepts every proposed mutation before it can affect the frontier:

```
1 if getattr(args, "hitl", True) and results:
2     for name in list(results.keys()):
3         energy_score = (metrics.get('total_input_tokens', 0) * 0.00001) + \
4                         (metrics.get('total_output_tokens', 0) * 0.00003) + \
5                         (bench_time * 0.1)
6
7         print(f"Avg Pass Rate: {_rate_str(avg)}")
8         print(f"Energy Score (lower is better): {energy_score:.2f}")
9         os.system("git diff .kota_skills/") # Show the mutation diff
10
11        resp = input("Approve this skill tree mutation? [y/N/feedback]: ")
12        if not resp.lower().startswith('y'):
13            del results[name]
14            os.system("git checkout -- .kota_skills/") # Rollback
```

Listing 6: HITL review gateway in meta_harness.py. The human reviews the git diff before approving any skill mutation.

The HITL reviewer sees the Energy Score, the pass rate, and the exact git diff of the skill file changes. This makes the review process analogous to a code review: the human approves or rejects a semantic diff to the agent’s behavioral specification.

5.3 The Energy Score

The Energy Score is a composite hardware proxy that penalizes energetically wasteful mutations:

$$E = \underbrace{T_{\text{in}} \cdot 0.00001}_{\text{input cost}} + \underbrace{T_{\text{out}} \cdot 0.00003}_{\text{output cost}} + \underbrace{t_{\text{wall}} \cdot 0.1}_{\text{latency cost}}$$

where T_{in} and T_{out} are the total input and output token counts across all trials, and t_{wall} is the benchmark wall-clock time in seconds. Output tokens are weighted three times higher than input tokens, reflecting the empirical observation that generation is more computationally expensive than prefilling. A skill tree that bloats the system prompt with verbose instructions will produce a higher energy score and be more likely to be rejected or pruned.

5.4 Multi-Model Support

The harness is designed to be model-agnostic. It has been tested with Claude 3.7 Sonnet (the primary frontier model), DeepSeek-V3, and GLM-4, all configured via a shared model registry. This supports the polyglot philosophy at the model level: the best model for the task is selected, not the most familiar one.

5.5 Continual Learning Ensembles

A key failure mode of evolutionary approaches is *catastrophic forgetting*: the frontier converges on a generalist agent that performs well on average but poorly on specific task categories it has “unlearned”. Kota mitigates this by maintaining the frontier as a K-NN dictionary:

```

1 # frontier[task_key] = {"best_agent": agent_name, "pass_rate": rate}
2 ensemble_tasks = [k for k in frontier.keys() if k != "_best"]
3 if ensemble_tasks:
4     ensemble_agents = set()
5     ensemble_passes = 0
6     for t in ensemble_tasks:
7         ensemble_agents.add(frontier[t].get("best_agent"))
8         ensemble_passes += frontier[t].get("pass_rate", 0)
9     ensemble_avg = ensemble_passes / len(ensemble_tasks)
10    print(f"Ensemble Pool: {len(ensemble_agents)} specialized agents")
11    print(f"Ensemble Avg Pass Rate (rounded): {_rate_str(ensemble_avg)}")

```

Listing 7: Ensemble pool simulation in meta_harness.py Phase Final.

The frontier acts as a *task-specialist ensemble*: for any given benchmark task, the harness routes it to the historically best-performing agent configuration. This is structurally equivalent to the Continual Learning approach of Schwarz et al. [5], where a population of specialist networks is maintained rather than a single continually updated network.

5.6 Agent-Native Documentation (OpenWiki)

As a repository evolves, proposer agents require structural context to propose mutations without breaking dependencies. However, reading the entire project directory tree and all source files on every generation cycle introduces massive token overhead and context rot.

To address this, Kota integrates an OpenWiki-style *agent-native context mapper* in **MetaHarness**. The mapper recursively walks the repository directory structure—automatically ignoring binary targets, test suites, and raw docs directories (e.g., `tests/`, `docs/`, `.git/`)—and builds a structured tree of the project. Simultaneously, it extracts module-level docstrings from all Python source files. This dense, token-efficient repository map is output directly to **AGENTS.md** at the root of the project.

To ensure the map remains synchronized with active development, we implement an automated CI/CD workflow (`.github/workflows/agent-docs-sync.yml`). On every push to the `main` branch, the workflow runs the documentation generator and automatically opens a Pull Request if any structural changes or docstring updates are detected. Proposer agents can parse **AGENTS.md** to instantly grasp the project architecture and module roles, maximizing evolution efficiency.

6 The LLM Wiki: Persistent Knowledge Compilation

The standard RAG paradigm treats knowledge as a static retrieval problem: embed documents, index them, retrieve relevant chunks at query time. This is computationally efficient but limits dynamic synthesis. The LLM re-derives its understanding of the corpus on every query. Cross-document syntheses are not preserved. Contradictions are not resolved proactively. Knowledge does not compound.

Kota introduces the *LLM Wiki* pattern as an alternative: the agent maintains a persistent, curated knowledge base (`.kota_knowledge/`) of interlinked Markdown files. The **Librarian** mode (an Option Keyboard chord) makes the agent the primary maintainer of this knowledge base.

6.1 Architecture

The LLM Wiki has three layers, analogous to the three layers of a software system:

1. **Raw Sources** (immutable): The original documents, papers, and data files. The Librarian reads from these but never modifies them.
2. **The Wiki** (`.kota_knowledge/`): The compiled, curated knowledge base. The Librarian owns this layer entirely.
3. **The Schema** (`.kota_skills/librarian.md`): The behavioral specification that governs how the Librarian operates: what operations it performs, in what order, and with what consistency guarantees.

6.2 Operations

The Librarian skill defines three core operations:

- **INGEST.** When asked to process a new source, the Librarian reads it, extracts key claims, writes a summary page, updates `index.md`, updates any relevant concept or entity pages, and appends to `log.md`. A single source may touch 10–15 pages.
- **QUERY.** The Librarian searches the wiki for relevant pages, reads them, synthesizes a response, and files the answer back into the wiki as a new page. High-quality analyses compound into the knowledge base rather than disappearing into chat history.
- **LINT.** Periodically, the Librarian health-checks the wiki: finding orphan pages, stale claims, missing cross-references, and unresolved contradictions.

6.3 The Index and Log

`index.md` is content-oriented: a catalog of all pages, their summaries, and their source counts. `log.md` is chronological: an append-only record of every operation with a consistent prefix format (`## [YYYY-MM-DD HH:MM:SS] Action | Description`) that makes it parseable with standard Unix tools.

6.4 Relation to Vannevar Bush’s Memex

The LLM Wiki is a practical realization of Vannevar Bush’s 1945 vision of the Memex [7]: a personal, actively curated knowledge store with associative trails between documents, where the connections between documents are as valuable as the documents themselves. Bush identified the bottleneck correctly: “The human mind does not work that way. It operates by association.” The bottleneck he could not solve was maintenance cost. The LLM Librarian eliminates this cost.

6.5 External Vault Support

The Librarian skill explicitly supports external knowledge base targets. Rather than constraining knowledge to `.kota_knowledge/`, the user can point the Librarian to an Obsidian vault, a Notion export directory, or a Google Drive mount. The architecture is the same; only the target path changes. This enables Kota to serve as the LLM backend for any Markdown-based personal knowledge management system.

7 Cognitive RF Security: Sensing, Diversity, and Information Theoretic Resilience

To secure Kota against context poisoning and adversarial prompt injection, we draw an explicit architectural parallel between wireless communication channels and Large Language

Model inference. By treating the agent’s context window as an RF spectrum and the prompt as a signal, we can apply rigorous mechanisms from telecommunications.

7.1 MIMO Prompting and Spatial Diversity

In wireless systems, Multiple-Input Multiple-Output (MIMO) technology leverages spatial diversity by transmitting signals across multiple physical paths, maximizing the Signal-to-Interference-plus-Noise Ratio (SINR). In Kota, we map this to an ensemble of sub-agents.

Let $\mathbf{x}$ be the user’s input prompt and $\mathbf{H}_i$ represent the transformation matrix of the i -th LLM sub-agent. The received output from sub-agent i is subject to adversarial injection noise $\mathbf{n}_i$:

$$\mathbf{y}_i = \mathbf{H}_i \mathbf{x} + \mathbf{n}_i \quad (1)$$

By splitting the execution across N sub-agents operating with varying constraints and temperatures, we achieve *prompt spatial diversity*. Even if a prompt injection ($\mathbf{n}_j$) successfully hijacks sub-agent j , the other agents $k \neq j$ may reject it. A synthesis function (beamforming) $f(\mathbf{y}_1, \dots, \mathbf{y}_N)$ aggregates the outputs, filtering out the anomalous injection spike as noise.

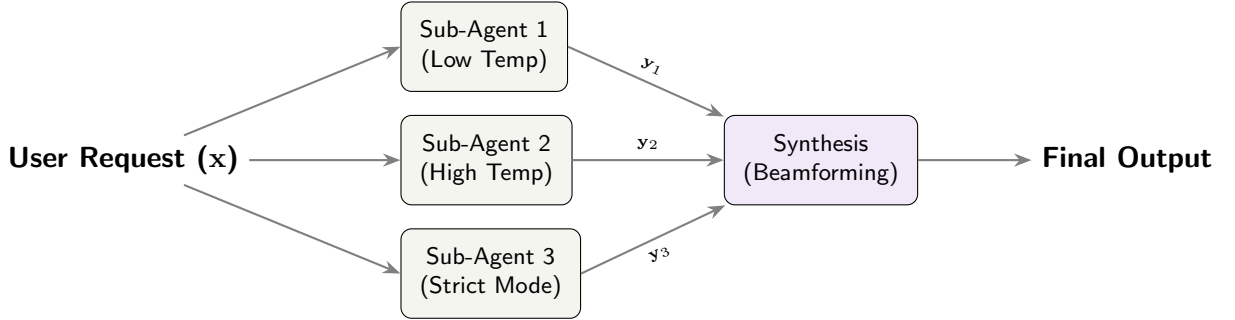


Figure 1: MIMO Prompting architecture leveraging Spatial Diversity.

7.2 Cognitive Context Sensing

Cognitive Radio (CR) devices actively monitor the RF spectrum using energy detection to identify interference. Signal presence is determined if the received energy $\Lambda(\mathbf{y})$ exceeds a threshold γ .

Kota implements a *Cognitive Context Radar*, a lightweight classification model acting as a sensing subsystem. Before execution, the radar sweeps the incoming context (e.g., fetched web data or file reads) for adversarial noise (high-perplexity anomalies or injection signatures).

$$\Lambda(\mathbf{c}) = \sum_j |c_j|^2 > \gamma \quad (2)$$

If the adversarial energy exceeds γ , Kota engages *Dynamic Spectrum Access*: it shifts from its permissive autonomous protocol to a highly restricted sandbox protocol, neutralizing the attack.

7.3 Markov Channels and Context Capacity

Following the framework of Markov Channels with General Inputs (Holliday, Goldsmith, and Glynn, 2002), we model the agent’s conversation history (working memory) as a discrete-time Markov chain S_t .

A prompt injection represents a *burst error* that floods the channel capacity, driving the mutual information $I(X;Y)$ between the user’s initial objective X and the generated output Y to zero.

$$C = \lim_{n \rightarrow \infty} \frac{1}{n} \max_{p(x^n)} I(X^n; Y^n) \quad (3)$$

Kota continuously monitors the mutual information/perplexity of its trace. A sudden drop signifies that channel capacity has been breached. In response, Kota initiates an ARQ-style *Context Reset*, flushing the compromised working memory and recovering a clean state from the Turso episodic memory database.

8 On Loops, Circuits, and Antifragility

We return to the foundational claim: loops and circuits are the same structure at different scales.

A **for** loop iterates over a collection, applying a transformation to each element. A simple LED circuit iterates over a voltage difference, applying a current-to-light conversion at each electron’s passage through the semiconductor junction. Both are directed graphs where energy flows through nodes that perform transformations. Both can be made arbitrarily complex: nested loops, circuit arrays, OLED matrices. Both require resources at every scale to function.

The Transformer architecture sits precisely at the intersection: it performs spectral transformations on the embedding space of input tokens, stepping the representational impedance of a sequence to enable long-range association. The attention mechanism is an energy-selective circuit: certain pathways (high-attention pairs) carry signal; others do not.

Antifragility [6] enters when the system is designed to respond constructively to stress. A rigid circuit breaks under voltage spikes; a current-limiting resistor absorbs them. A rigid agent fails catastrophically on out-of-distribution inputs; an Option Keyboard absorbs them by composing the most relevant skills dynamically. The evolutionary Meta-Harness is a stress-test mechanism: it deliberately applies adversarial pressure (novel TB2 tasks) to the skill tree and selects for configurations that survive.

The Energy Score closes the loop (literally): it connects the abstract performance of the agent to the physical cost of computation, ensuring that the system remains conscious of the energetic exchange at every level of the circuit.

9 Discussion and Future Work

Skill Tree as a Directed Acyclic Graph. Currently, skills are composed via linear weighting. A natural extension is to define skills as nodes in a DAG, where information flows through specialized processing stages (analogous to logic gates) before reaching the final prompt synthesis. This would enable conditional skill activation and richer compositional structures.

Spectral Memory Retrieval. The current episodic memory system uses exact keyword matching. A more principled approach would apply spectral analysis (e.g., SVD decomposition of the message embedding matrix) to retrieve memories based on semantic similarity rather than token overlap. This aligns with the spectral filtering approach of Agarwal et al. [8] for efficient sequence modeling.

Kota as a Knowledge Compiler. The LLM Wiki pattern suggests a research agenda: can an agent maintain a consistent, contradiction-free knowledge base across thousands of ingested documents? What consistency guarantees can be provided? How does the quality of the compiled wiki compare to human-curated resources like Tolkien Gateway or Wikipedia?

Robotic Extension. The architecture maps naturally onto embodied systems. The skill tree becomes a library of motor policies (in the DeepMind Robotics sense [10]); the episodic memory stores trajectories rather than text; the Meta-Harness evaluates physical task completion rather than software benchmarks. The energetic cost of the Energy Score becomes literal: joules consumed per task completed.

10 Conclusion

Kota is a working demonstration that the principles governing electrical circuits, neural circuits, software loops, and physical transformer architectures are not analogies; they are the same structure instantiated in different **substrates**. By engineering across these levels of abstraction (from Rust memory ownership to SQL episodic traces to Python evolutionary pressure to Markdown-compiled knowledge), Kota achieves the structural persistence required for an agent that genuinely grows stronger through use.

The system is deliberately incomplete. The skill tree will continue to evolve. The episodic memory will accumulate. The wiki will compound. The harness will apply pressure. That is the point.

References

- [1] Vaswani, A., et al. (2017). *Attention Is All You Need*. NeurIPS 2017.
- [2] Yao, S., et al. (2023). *ReAct: Synergizing Reasoning and Acting in Language Models*. ICLR 2023.
- [3] Barreto, A., et al. (2019). *The Option Keyboard: Combining Skills in Reinforcement Learning*. NeurIPS 2019.
- [4] Lampinen, A. K., et al. (2021). *Towards mental time travel: a hierarchical memory for reinforcement learning agents*. arXiv:2105.14039.
- [5] Schwarz, J., et al. (2018). *Progress & Compress: A scalable framework for continual learning*. ICML 2018.
- [6] Taleb, N. N. (2012). *Antifragile: Things That Gain from Disorder*. Random House.
- [7] Bush, V. (1945). *As We May Think*. The Atlantic Monthly, 176(1), 101–108.
- [8] Agarwal, N., et al. (2023). *Spectral State Space Models*. arXiv:2312.06837.
- [9] MIT EECS (2024). *6.102: Software Construction*. Massachusetts Institute of Technology.
- [10] Google DeepMind Robotics (2024). *Google Brain & Princeton Robotics Research*. <https://sites.google.com/view/gbrainprinceton/home>